

SMART INDIA HACKATHON 2026



Project Report

Problem Statement ID	SIH26027
Problem Statement Title	AI-Powered Automatic Block Planning to Maximize Asset Availability for Train Operations on Indian Railways
Theme	Transportation & Logistics
PS Category	Software
Team Name	TRIKAAL
Team ID	137710
Live Prototype	https://raksha-block-trikaal.vercel.app/
GitHub Repository	https://github.com/khushhh-14/Raksha_Block_Trikaal

ABSTRACT

Indian Railways runs one of the densest mixed-traffic networks in the world. Engineering (P-Way), Signal & Telecommunication (S&T), and Traction Distribution (TRD) departments each need to take the track out of service through a “block” to carry out maintenance. Today this coordination is manual — paper requisitions, phone calls, spreadsheets — so the same corridor is often blocked more than once for work that could have shared a single window, causing avoidable loss of traffic capacity, cascading delays, and a weak safety audit trail.

RAKSHA-BLOCK is a full-stack, AI-assisted block-planning and corridor-maintenance coordination platform built by Team TRIKAAL for SIH26027. It digitises the entire block-request lifecycle for all three departments and the Section Controller, and adds a Google OR-Tools CP-SAT solver that automatically bundles compatible requests into one shared possession window, a machine-learning risk-scoring service that prioritises the most urgent defects, a General & Subsidiary Rules (GSR) safety-compliance engine that blocks unsafe approvals, and a cryptographically hash-chained audit ledger for full traceability. On the deployed working prototype, this combination directly increases track availability for revenue trains while giving Section Controllers one safety-verified source of truth.

Table of Contents

1. Problem Understanding.....	3
2. Proposed Solution : RAKSHA-BLOCK.....	3
3. Website Flow.....	5
4. Dashboard & Functioning Screenshots.....	6
5. Expected Impact.....	7
6. Conclusion & Future Scope.....	8
7. Project Resources.....	8
References.....	8

1. PROBLEM UNDERSTANDING

Maintenance work — track tamping, rail grinding, OHE repair, signal interlocking changes, bridge repair, level-crossing gate work — requires the line to be closed to traffic for a “block.” Engineering, S&T, and TRD each raise block demand independently and largely in isolation, which creates the following core problems:

- Fragmented, paper- and phone-based requests with no shared visibility across departments.
- The same section is frequently blocked more than once for work that could be bundled into one “shadow block,” wasting traffic-carrying capacity.
- Section Controllers manually cross-check headways, traction isolation, and signal-interlocking rules under time pressure — a real risk of a safety rule being missed.
- No systematic way to prioritise which defects most urgently need a block.
- Train-delay impact is estimated informally, if at all, before a block is sanctioned, and decisions are not recorded in a tamper-evident way.

Team TRIKAAL interpreted the SIH26027 objective — AI-powered automatic block planning to maximise asset availability — as three connected goals: unify the block-request lifecycle across departments, use optimisation and ML to bundle work and prioritise risk automatically, and enforce railway safety rules with a verifiable audit trail throughout.

2. PROPOSED SOLUTION : RAKSHA-BLOCK

RAKSHA-BLOCK is a single-page web application covering the complete block lifecycle — submit, review, approve/modify/reject, bundle, publish, execute, and safety-clear — across Engineering, S&T, TRD, and the Section Controller, kept in sync locally and, when configured, in real time through Supabase.

2.1 Why RAKSHA-BLOCK Wins? — Unique Value Propositions

- **AI-assisted shadow-block bundling:** a Google OR-Tools CP-SAT solver finds compatible requests across departments and proposes one consolidated possession instead of several, reporting block-hours saved and conflicts avoided for every run — this is the direct, measurable lever on asset availability the problem statement asks for.
- **Safety enforced by code, not memory:** a 5-rule GSR compliance engine (headway, traction isolation, interlocking, shadow-bundling alignment, speed-restriction caution) automatically blocks an unsafe approval and tells the controller exactly what to fix.
- **Tamper-evident audit trail:** every decision is written to a SHA-256 hash-chained “Integrity Ledger” that can be cryptographically verified end-to-end in one click — not just logged, but provably unaltered.
- **ML-based risk prioritisation:** defects are ranked CRITICAL/HIGH/MEDIUM/LOW by a LightGBM model with explainable drivers, so the highest-risk assets are never crowded out by low-priority work.
- **Impact is quantified before approval:** every request shows live passenger-delay, freight-delay, and rerouted/held-train estimates, so a controller sees the operational cost of a block before sanctioning it.

- **One data model, three synchronised views:** a Gantt “Maintenance Planning” board, a time-distance String Chart, and a live corridor map all render from the same schedule — nothing is re-entered or reconciled by hand.
- **Real-time, cross-device, and field-ready:** Supabase realtime sync with a polling fallback, plus responsive layouts and audio alerts, make it usable in the control room and on a mobile device.

2.2. FEATURE HIGHLIGHTS

Department Officer (Engineering / S&T / TRD)

- **Structured request form** (section, chainage, work category, machinery, date/time, priority) with a live train-impact estimate shown before submission.
- **Status tracking** (Pending → Approved / Modified & Approved / Rejected → Completed) and shadow-block discovery to encourage voluntary bundling.
- **Safety Checkout:** a mandatory 3-point checklist certifies Line Clear before a completed block can close.

Section Controller (Admin)

- **Unified review queue** across all departments, with GSR Compliance Panel, AI Co-Pilot (Gemini-powered “Raksha-Saarthi” chat), and one-click AI Optimizer (CP-SAT bundling with block-hours-saved and conflict-reduction metrics).
- **Approve / Modify / Reject** with a reasoned remark logged to the Integrity Ledger, then Publish the coordinated schedule.
- **Visual command centre:** Gantt Maintenance Planning board, String/Marey Chart, Live Analytics Map, Impact KPI dashboard (“Manual Planning vs. RAKSHA-BLOCK”), and the hash-chained Integrity Ledger with chain verification.

Platform-Wide

- **Role-based login,** Supabase realtime cross-device sync (5s polling fallback), Indian Railways zone/division context (NR/WR/CR/ER/SR), CSV/PDF export, and control-room-ready responsive UI with audio alerts.

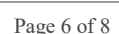
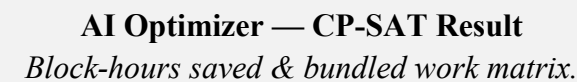
3. WEBSITE FLOW

- Login — officer/controller signs in by role (Engineering / S&T / TRD / Admin).
- Officer submits a request via the New Request form, with a live train-impact estimate; it appears instantly in the Admin review queue.
- Controller opens the request; the GSR Compliance Panel auto-checks safety rules, and the AI Optimizer proposes bundled windows (CP-SAT), optionally assisted by the AI Co-Pilot chat.
- Controller approves/modifies/rejects (logged to the Integrity Ledger) and publishes the coordinated schedule — visible on the Gantt board and String Chart, with the officer notified.
- After execution, the officer completes Safety Checkout; the controller reviews the Live Analytics Map, Impact KPI dashboard, and Integrity Ledger for oversight.

3.1. TECHNICAL ARCHITECTURE & TECH STACK

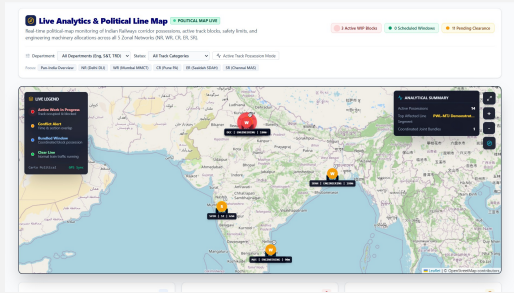
Layer	Technology
Frontend	React 19 + TypeScript, Vite 6, Tailwind CSS 4
Maps / Charts	Leaflet, custom SVG Gantt & String Chart, Recharts (KPI trends)
Export / Icons / Motion	jsPDF, jsPDF-AutoTable, html2canvas, Lucide React; Motion
Auth	bcryptjs-hashed demo role credentials
Realtime data	Supabase (PostgreSQL + Realtime), local-storage fallback
AI Co-Pilot	Google Gemini API (@google/genai), local deterministic fallback
Optimisation engine	Python 3.10+, Google OR-Tools CP-SAT (FastAPI service)
ML risk scoring	LightGBM model served via Python API, local fallback
Backend / DevOps	Node.js, Express, Docker & Docker Compose, Nginx
Deployment / VCS	Vercel, Git / GitHub

Login Portal
Four role: Engineering, S&T, TRD, Admin.



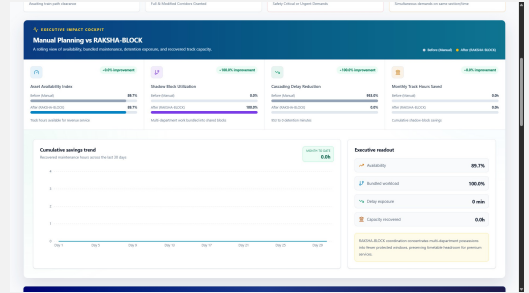
Live Analytics Map Dashboard

Zone-wise possession & conflict status.



Impact KPI Dashboard

Manual Planning vs. RAKSHA-BLOCK



Integrity Ledger

Hash-chained audit trail, chain verified.

ID	Request ID	Requester	Request Type	Request Status	Request Date	Request Details
1	REQ-001-2024-001	RAKSHA-BLOCK	TRACK	PENDING	2024-01-01	Request for track availability
2	REQ-002-2024-002	RAKSHA-BLOCK	TRACK	PENDING	2024-01-02	Request for track availability
3	REQ-003-2024-003	RAKSHA-BLOCK	TRACK	PENDING	2024-01-03	Request for track availability
4	REQ-004-2024-004	RAKSHA-BLOCK	TRACK	PENDING	2024-01-04	Request for track availability
5	REQ-005-2024-005	RAKSHA-BLOCK	TRACK	PENDING	2024-01-05	Request for track availability
6	REQ-006-2024-006	RAKSHA-BLOCK	TRACK	PENDING	2024-01-06	Request for track availability
7	REQ-007-2024-007	RAKSHA-BLOCK	TRACK	PENDING	2024-01-07	Request for track availability
8	REQ-008-2024-008	RAKSHA-BLOCK	TRACK	PENDING	2024-01-08	Request for track availability
9	REQ-009-2024-009	RAKSHA-BLOCK	TRACK	PENDING	2024-01-09	Request for track availability
10	REQ-010-2024-010	RAKSHA-BLOCK	TRACK	PENDING	2024-01-10	Request for track availability

Safety Checkout Modal

3-point Line Clear certification.

5. EXPECTED IMPACT

By consolidating multi-department block demand into fewer, jointly protected windows, RAKSHA-BLOCK targets a measurable increase in track availability for revenue services, lower bundled-workload disruption, and reduced cumulative passenger/freight delay exposure — tracked live on the platform's own Impact KPI Dashboard. The prototype runs on clearly labelled synthetic data for a hackathon demo; a production rollout would connect to Indian Railways' live TMS/COA/SMMS/TDMS systems, for which dedicated integration adapters already exist in the codebase (src/integrations/), alongside migrating demo credentials to Supabase Auth with per-department Row-Level Security.

6. CONCLUSION & FUTURE SCOPE

RAKSHA-BLOCK shows that the manual, siloed block-planning process on Indian Railways can be replaced by a single, safety-rule-aware, AI-assisted digital workflow without sacrificing auditability — covering the full lifecycle from request to safety clearance on a working, deployed prototype. Planned next steps: Supabase Auth with per-department RLS, persisted AI/optimizer decision history, configurable notification preferences, multi-day recurring block windows, and live TMS/COA/SMMS/TDMS integration.

7. PROJECT RESOURCES

- **Live Prototype:** <https://raksha-block-trikaal.vercel.app/>
- **GitHub Repository:** https://github.com/khushhh-14/Raksha_Block_Trikaal
- **Problem Understanding Video:** <https://youtu.be/xhBWHfrbTVQ>

REFERENCES

- [1] Smart India Hackathon 2026, Problem Statement SIH26027 — “AI-Powered Automatic Block Planning to Maximize Asset Availability for Train Operations on Indian Railways,” Government of India. <https://www.sih.gov.in/>
- [2] Google OR-Tools Documentation — CP-SAT Solver. https://developers.google.com/optimization/cp/cp_solver
- [2] Supabase Documentation - Realtime Database & Postgres. <https://supabase.com/docs>
- [3] React Documentation. <https://react.dev/>
- [5] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” NeurIPS, 2017.
- [6] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” NeurIPS, 2017.
- [7] Google Gemini API Documentation. <https://ai.google.dev/>
- [8] Indian Railways General & Subsidiary Rules (GSR), Railway Board, Government of India.